

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: C

Batch: 3

Practical No: 4

Roll no: 59

PRN No: 12413745

Name of Student: Netra Lalwani

Subject Name: Deep Learning

Problem Statement:

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm:

1. Load the Dataset

- Collect tomato leaf images from the Tomato Disease dataset.
- Organize images into folders according to their disease classes.

2. Preprocess the Images

- Resize all images to a fixed size such as $128 \times 128 \times 3$.
- Normalize pixel values from $[0, 255]$ to $[0, 1]$.
- Assign a numerical label to each disease class.

3. Split the Dataset

- Divide the dataset into training and testing sets, e.g.:
 - 80% Training
 - 20% Testing.

4. Build the CNN Model

- Add a **Convolutional Layer** to extract features such as edges, shapes, and textures.
- Apply **ReLU activation**.
- Add a **Max Pooling Layer** to reduce spatial dimensions.
- Repeat convolution and pooling layers to learn higher-level features.

- Flatten the extracted feature maps.
- Add one or more **Fully Connected (Dense) Layers**.
- Add a **Softmax output layer** with one neuron for each disease class.

5. **Compile the Model**

- Use **Adam optimizer**.
- Use **Categorical Cross-Entropy** as the loss function.
- Use **Accuracy** as the evaluation metric.

6. **Train the CNN**

- Feed the training images and their labels into the CNN.
- Perform forward propagation to obtain predictions.
- Calculate the loss between predicted and actual labels.
- Perform backpropagation to update the weights.
- Repeat for a fixed number of epochs.

7. **Evaluate the Model**

- Test the trained model using unseen test images.
- Calculate classification accuracy.
- Generate a confusion matrix and other metrics if required.

8. **Predict Disease**

- Give a new tomato leaf image to the trained CNN.
- Preprocess it in the same way as the training images.
- Pass it through the CNN.
- The class with the highest Softmax probability is the predicted disease

Code:

```

import os, random, pathlib, itertools, textwrap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing.image import load_img, img_to_array

from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 110

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices('GPU'))

```

```

TensorFlow version: 2.20.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

```

```

import kagglehub

# Download latest version
path = kagglehub.dataset_download("muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten")
print("Path to dataset files:", path)

```

```

Downloading to /root/.cache/kagglehub/datasets/muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten/1.archive...
100%|██████████| 4.77G/4.77G [03:53<00:00, 21.9MB/s]Extracting files...

```

```

Path to dataset files: /root/.cache/kagglehub/datasets/muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten/versions/1

```

```

data_root = pathlib.Path(path)

for p in sorted(data_root.rglob('*'))[:30]:
    depth = len(p.relative_to(data_root).parts)
    print(" " * depth + p.name)

```

Fruit And Vegetable Diseases Dataset

```

Apple__Healthy
  FreshApple (1).jpg
  FreshApple (10).jpg
  FreshApple (100).jpg
  FreshApple (101).jpg
  FreshApple (102).jpg
  FreshApple (103).jpg
  FreshApple (104).jpg
  FreshApple (105).jpg
  FreshApple (106).jpg
  FreshApple (107).jpg
  FreshApple (108).jpg
  FreshApple (109).jpg

```

```

def find_class_dirs(root):
    """Return the directory whose immediate children are class folders full of images."""
    candidates = []
    for d in root.rglob('*'):
        if d.is_dir():
            subdirs = [c for c in d.iterdir() if c.is_dir()]
            if len(subdirs) >= 2:
                sample = subdirs[0]
                imgs = list(sample.glob('*.jpg')) + list(sample.glob('*.png')) + list(sample.glob('*.jpeg'))
                if imgs:
                    candidates.append(d)
    candidates.sort(key=lambda x: len(x.parts))
    return candidates[0] if candidates else root

DATA_DIR = find_class_dirs(data_root)
print("Using data directory:", DATA_DIR)
print("Classes found:", sorted([d.name for d in DATA_DIR.iterdir() if d.is_dir()]))

```

```

Using data directory: /root/.cache/kagglehub/datasets/muhammad0subhan/fruit-and-vegetable-disease-healthy-vs-rotten/versions/1/Fruit And Vegi
Classes found: ['Apple__Healthy', 'Apple__Rotten', 'Banana__Healthy', 'Banana__Rotten', 'Bellpepper__Healthy', 'Bellpepper__Rotten', 'Carrot_

```

```

class_dirs = sorted([d for d in DATA_DIR.iterdir() if d.is_dir()])
class_names = [d.name for d in class_dirs]
IMG_EXTS = ('*.jpg', '*.jpeg', '*.png', '*.JPG', '*.JPEG', '*.PNG')

counts = {}
for d in class_dirs:
    n = sum(len(list(d.glob(ext))) for ext in IMG_EXTS)
    counts[d.name] = n

df_counts = pd.DataFrame(sorted(counts.items(), key=lambda x: -x[1]), columns=["class", "count"])
print(f"Total classes: {len(class_names)}")
print(f"Total images: {df_counts['count'].sum()}")
df_counts

```

Total classes: 28
Total images: 29277

	class	count
0	Apple__Rotten	2925
1	Banana__Rotten	2797
2	Apple__Healthy	2438
3	Mango__Rotten	2247
4	Orange__Rotten	2186
5	Orange__Healthy	2075
6	Banana__Healthy	1999
7	Mango__Healthy	1813
8	Strawberry__Healthy	1603



Visualization 1: Class distribution

```

fig, ax = plt.subplots(figsize=(10, max(4, 0.35 * len(df_counts))))
sns.barplot(data=df_counts, y="class", x="count", palette="viridis", ax=ax)
ax.set_title("Number of Images per Class", fontsize=14, weight="bold")
ax.set_xlabel("Image count")
ax.set_ylabel("")
for i, v in enumerate(df_counts["count"]):
    ax.text(v + 2, i, str(v), va="center", fontsize=8)
plt.tight_layout()
plt.show()

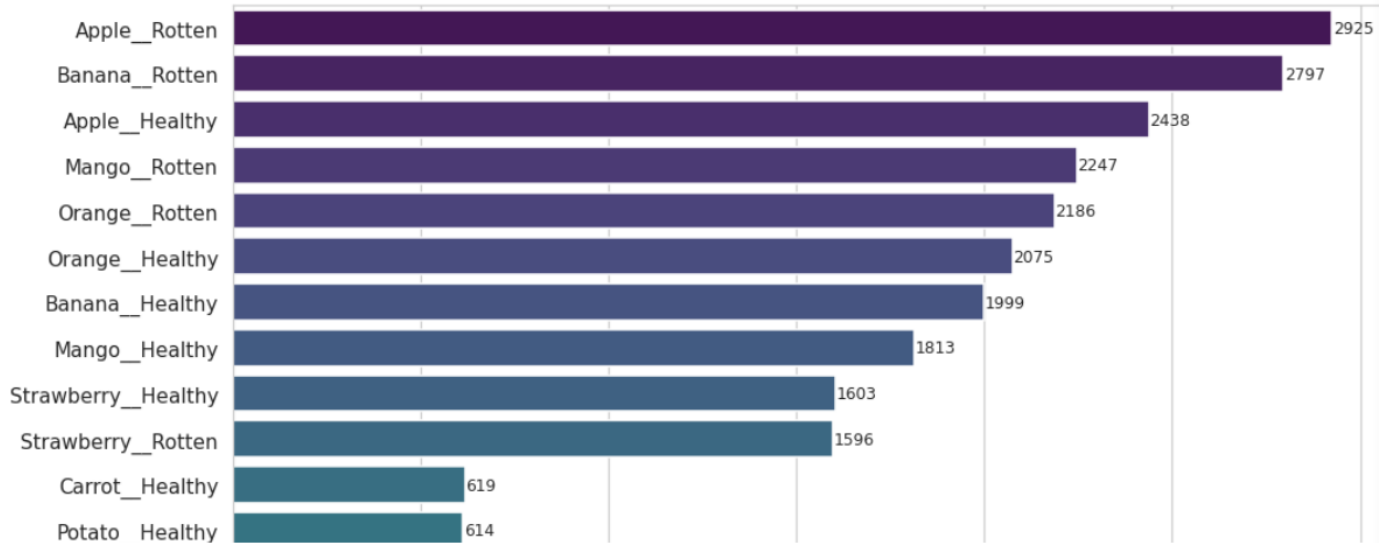
```

```

imbalance_ratio = df_counts["count"].max() / df_counts["count"].min()
print(f"Imbalance ratio (largest class / smallest class): {imbalance_ratio:.2f}x")

```

Number of Images per Class



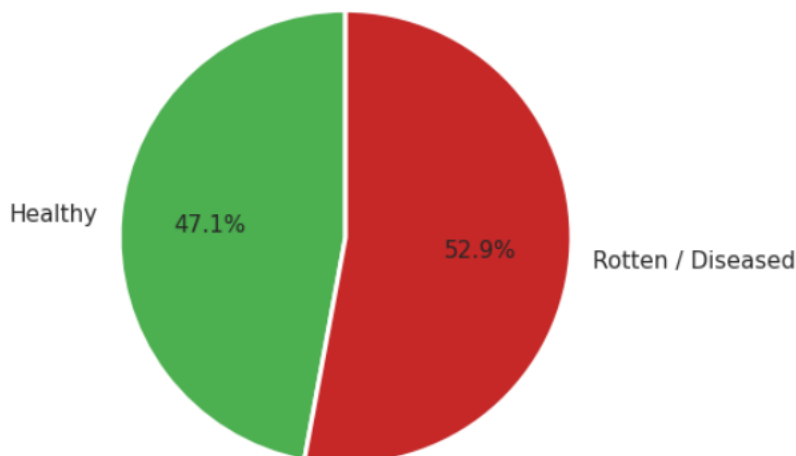
Visualization 2: Healthy vs Rotten aggregate split

```
def label_group(name):
    n = name.lower()
    if "rotten" in n or "disease" in n or "damaged" in n:
        return "Rotten / Diseased"
    return "Healthy"
```

```
df_counts["group"] = df_counts["class"].apply(label_group)
group_totals = df_counts.groupby("group")["count"].sum()
```

```
fig, ax = plt.subplots(figsize=(5, 5))
colors = ["#4CAF50", "#C62828"]
ax.pie(group_totals, labels=group_totals.index, autopct="%1.1f%",
        colors=colors, startangle=90, wedgeprops={"edgecolor": "white", "linewidth": 2})
ax.set_title("Overall Healthy vs Rotten/Diseased Split", fontsize=13, weight="bold")
plt.tight_layout()
plt.show()
```

Overall Healthy vs Rotten/Diseased Split



```

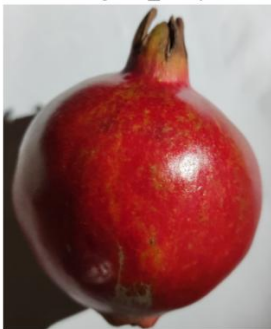
# Visualization 3: Sample image grid
sample_classes = random.sample(class_names, min(8, len(class_names)))

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
for ax, cls in zip(axes.flatten(), sample_classes):
    cls_dir = DATA_DIR / cls
    imgs = []
    for ext in IMG_EXTS:
        imgs.extend(list(cls_dir.glob(ext)))
    img_path = random.choice(imgs)
    img = load_img(img_path)
    ax.imshow(img)
    ax.set_title(textwrap.fill(cls, 20), fontsize=9)
    ax.axis("off")
plt.suptitle("Sample Images Across Classes", fontsize=15, weight="bold")
plt.tight_layout()
plt.show()

```

Sample Images Across Classes

Pomegranate_Healthy



Cucumber_Healthy

Banana_Rotten



Apple_Healthy



Potato_Healthy

Potato_Rotten



Bellpepper_Healthy

Cucumber_Healthy



Carrot_Rotten



Potato_Healthy



Bellpepper_Healthy

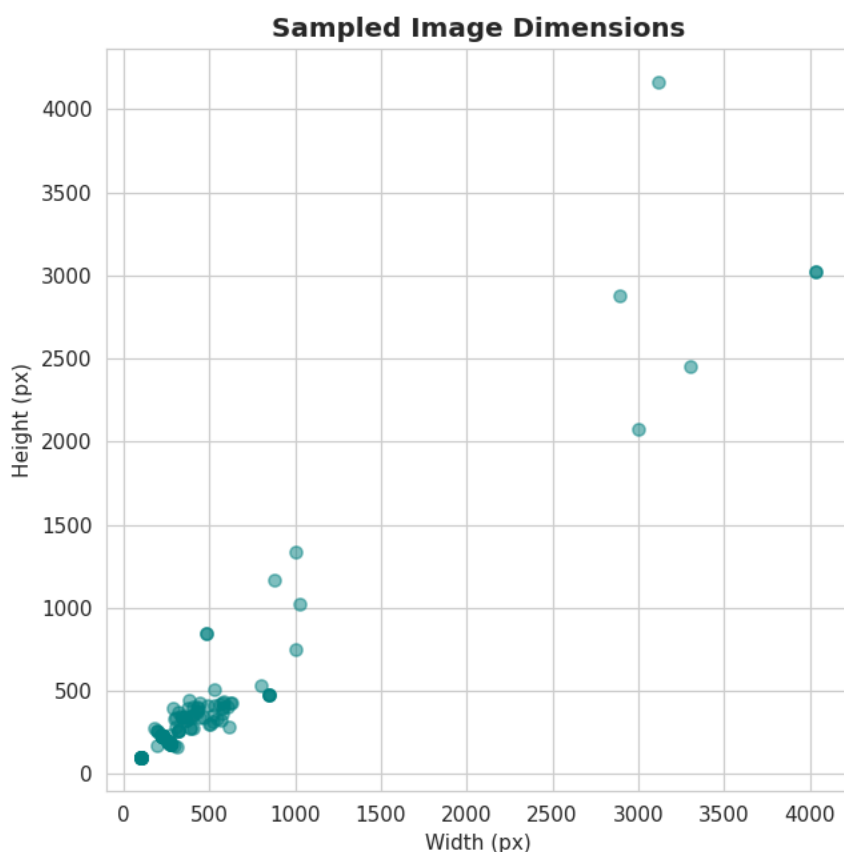


```

# Visualization 4: Image dimension distribution
sizes = []
for cls in class_names[:10]:
    cls_dir = DATA_DIR / cls
    imgs = []
    for ext in IMG_EXTS:
        imgs.extend(list(cls_dir.glob(ext)))
    for p in random.sample(imgs, min(15, len(imgs))):
        with load_img(p) as im:
            sizes.append(im.size)

widths, heights = zip(*sizes)
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(widths, heights, alpha=0.5, color="teal")
ax.set_xlabel("Width (px)")
ax.set_ylabel("Height (px)")
ax.set_title("Sampled Image Dimensions", fontsize=13, weight="bold")
plt.tight_layout()
plt.show()
print(f"Median width: {np.median(widths):.0f}px, Median height: {np.median(heights):.0f}px")

```



```

) IMG_SIZE = (160, 160)
  BATCH_SIZE = 32

raw_train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.3, subset="training", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode="int"
)

raw_val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.3, subset="validation", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode="int"
)

class_names = raw_train_ds.class_names
num_classes = len(class_names)
print("Classes:", class_names)

val_batches = tf.data.experimental.cardinality(raw_val_ds)
test_ds = raw_val_ds.take(val_batches // 2)
val_ds = raw_val_ds.skip(val_batches // 2)
train_ds = raw_train_ds

print(f"Train batches: {tf.data.experimental.cardinality(train_ds).numpy()}")
print(f"Val batches: {tf.data.experimental.cardinality(val_ds).numpy()}")
print(f"Test batches: {tf.data.experimental.cardinality(test_ds).numpy()}")

```

```

Found 29277 files belonging to 28 classes.
Using 20494 files for training.
Found 29277 files belonging to 28 classes.
Using 8783 files for validation.
Classes: ['Apple_Healthy', 'Apple_Rotten', 'Banana_Healthy', 'B
Train batches: 641
Val batches: 138
Test batches: 137

```

```

: data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.15),
    layers.RandomContrast(0.15),
], name="data_augmentation")

rescale = layers.Rescaling(1.0 / 255)
AUTOTUNE = tf.data.AUTOTUNE

def prepare(ds, training=False):
    if training:
        ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y), num_parallel_calls=AUTOTUNE)
        ds = ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE)
        return ds.cache().prefetch(buffer_size=AUTOTUNE)

train_ds_p = prepare(train_ds, training=True)
val_ds_p = prepare(val_ds, training=False)
test_ds_p = prepare(test_ds, training=False)

```

```
# Visualization 5: Effect of augmentation
for images, labels in train_ds.take(1):
    example_img = images[0]
    break

fig, axes = plt.subplots(1, 5, figsize=(16, 4))
axes[0].imshow(example_img.numpy().astype("uint8"))
axes[0].set_title("Original")
axes[0].axis("off")
for i in range(1, 5):
    aug_img = data_augmentation(tf.expand_dims(example_img, 0), training=True)[0]
    axes[i].imshow(aug_img.numpy().astype("uint8"))
    axes[i].set_title(f"Augmented #{i}")
    axes[i].axis("off")
plt.suptitle("Data Augmentation Examples", fontsize=14, weight="bold")
plt.tight_layout()
plt.show()
```

Data Augmentation Examples



```
# Class weights to handle imbalance
name_to_count = dict(zip(df_counts["class"], df_counts["count"]))
counts_in_order = np.array([name_to_count.get(c, 1) for c in class_names])
class_weight = {i: (counts_in_order.sum() / (num_classes * c)) for i, c in enumerate(counts_in_order)}
print("Class weights:")
for i, cls in enumerate(class_names):
    print(f" {cls:35s} -> {class_weight[i]:.2f}")
```

```
Class weights:
Apple_Healthy          -> 0.43
Apple_Rotten           -> 0.36
Banana_Healthy         -> 0.52
Banana_Rotten          -> 0.37
Bellpepper_Healthy     -> 1.71
Bellpepper_Rotten      -> 1.77
Carrot_Healthy         -> 1.69
Carrot_Rotten          -> 1.81
Cucumber_Healthy       -> 1.72
Cucumber_Rotten        -> 1.76
Grape_Healthy          -> 5.23
Grape_Rotten           -> 5.23
Guava_Healthy          -> 5.23
Guava_Rotten           -> 5.23
Jujube_Healthy         -> 5.23
Jujube_Rotten          -> 5.23
Mango_Healthy          -> 0.58
Mango_Rotten           -> 0.47
Orange_Healthy         -> 0.50
Orange_Rotten          -> 0.48
Pomegranate_Healthy    -> 5.23
Pomegranate_Rotten     -> 5.23
Potato_Healthy         -> 1.70
Potato_Rotten          -> 1.70
```

```
def build_cnn(input_shape, num_classes):
    inputs = keras.Input(shape=input_shape)

    x = layers.Conv2D(32, 3, padding="same", activation="relu")(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(256, 3, padding="same", activation="relu", name="last_conv")(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D()(x)

    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256, activation="relu")(x)
    x = layers.Dropout(0.4)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    return keras.Model(inputs, outputs, name="custom_disease_cnn")

cnn_model = build_cnn(IMG_SIZE + (3,), num_classes)
cnn_model.summary()
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 160, 160, 3)	0
conv2d (Conv2D)	(None, 160, 160, 32)	896
batch_normalization (BatchNormalization)	(None, 160, 160, 32)	128
conv2d_1 (Conv2D)	(None, 160, 160, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 80, 80, 32)	0
conv2d_2 (Conv2D)	(None, 80, 80, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 80, 80, 64)	256
conv2d_3 (Conv2D)	(None, 80, 80, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 40, 40, 64)	0
conv2d_4 (Conv2D)	(None, 40, 40, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 40, 40, 128)	512
conv2d_5 (Conv2D)	(None, 40, 40, 128)	147,584
max_pooling2d_2 (MaxPooling2D)	(None, 20, 20, 128)	0
last_conv (Conv2D)	(None, 20, 20, 256)	295,168

batch_normalization_3 (BatchNormalization)	(None, 20, 20, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 10, 10, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 256)	65,792
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 28)	7,196

Total params: 657,084 (2.51 MB)
Trainable params: 656,124 (2.50 MB)
Non-trainable params: 960 (3.75 KB)

```

IMG_SIZE = (64, 64)      # much smaller = much faster
BATCH_SIZE = 64

raw_train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.3, subset="training", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode="int"
)
raw_val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.3, subset="validation", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode="int"
)

class_names = raw_train_ds.class_names
num_classes = len(class_names)

val_batches = tf.data.experimental.cardinality(raw_val_ds)
test_ds = raw_val_ds.take(val_batches // 2)
val_ds = raw_val_ds.skip(val_batches // 2)
train_ds = raw_train_ds

rescale = layers.Rescaling(1.0 / 255)
AUTOTUNE = tf.data.AUTOTUNE

# no augmentation for now – it's extra compute we don't need for a quick run
train_ds_p = train_ds.map(lambda x, y: (rescale(x), y)).take(60).cache().prefetch(AUTOTUNE)
val_ds_p   = val_ds.map(lambda x, y: (rescale(x), y)).take(20).cache().prefetch(AUTOTUNE)
test_ds_p  = test_ds.map(lambda x, y: (rescale(x), y)).cache().prefetch(AUTOTUNE)

```

```
def build_cnn_fast(input_shape, num_classes):
    inputs = keras.Input(shape=input_shape)
    x = layers.Conv2D(16, 3, padding="same", activation="relu")(inputs)
    x = layers.MaxPooling2D()(x)
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Conv2D(64, 3, padding="same", activation="relu", name="last_conv")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inputs, outputs, name="custom_disease_cnn_fast")

cnn_model = build_cnn_fast(IMG_SIZE + (3,), num_classes)

cnn_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = cnn_model.fit(
    train_ds_p,
    validation_data=val_ds_p,
    epochs=5
)
```

```
Found 29277 files belonging to 28 classes.
Using 20494 files for training.
Found 29277 files belonging to 28 classes.
Using 8783 files for validation.
Epoch 1/5
60/60 ----- 45s 655ms/step - accuracy: 0.0812 - loss: 3.1108 - val_accuracy: 0.0922 - val_loss: 2.9596
Epoch 2/5
60/60 ----- 1s 11ms/step - accuracy: 0.1307 - loss: 2.8887 - val_accuracy: 0.1484 - val_loss: 2.7673
Epoch 3/5
60/60 ----- 1s 9ms/step - accuracy: 0.1844 - loss: 2.7211 - val_accuracy: 0.2297 - val_loss: 2.6036
Epoch 4/5
60/60 ----- 1s 9ms/step - accuracy: 0.2253 - loss: 2.5791 - val_accuracy: 0.2906 - val_loss: 2.4414
Epoch 5/5
60/60 ----- 0s 8ms/step - accuracy: 0.2573 - loss: 2.4371 - val_accuracy: 0.3164 - val_loss: 2.2987
```

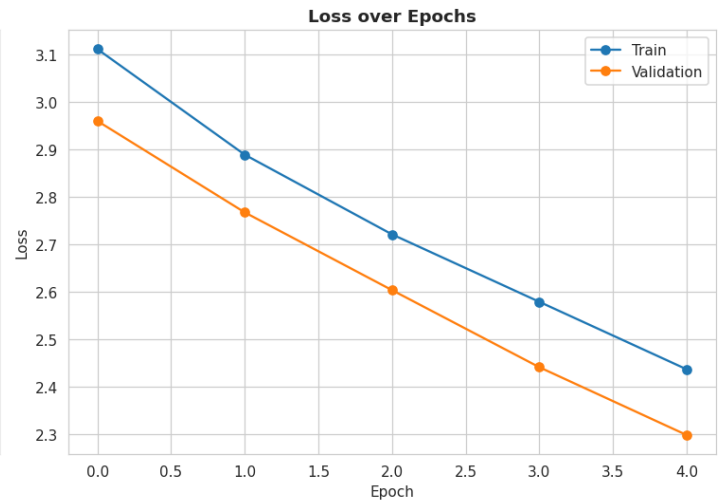
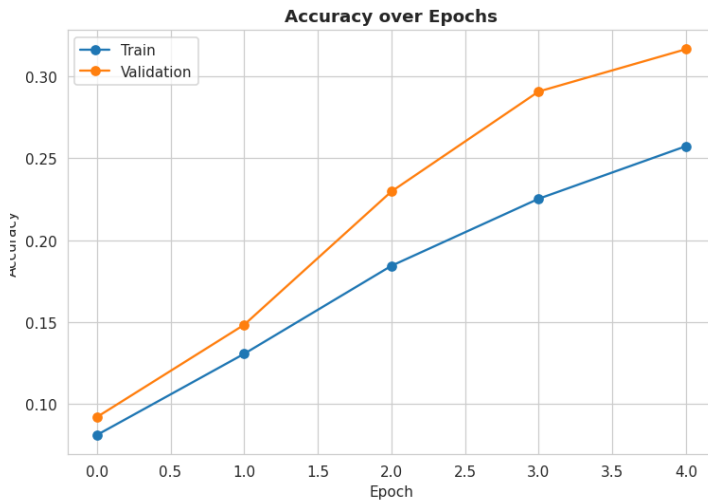
Visualization 6: Training curves

```
hist_df = pd.DataFrame(history.history)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(hist_df["accuracy"], label="Train", marker="o")
axes[0].plot(hist_df["val_accuracy"], label="Validation", marker="o")
axes[0].set_title("Accuracy over Epochs", weight="bold")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Accuracy"); axes[0].legend()

axes[1].plot(hist_df["loss"], label="Train", marker="o")
axes[1].plot(hist_df["val_loss"], label="Validation", marker="o")
axes[1].set_title("Loss over Epochs", weight="bold")
axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Loss"); axes[1].legend()
```

```
plt.tight_layout()
plt.show()
```



```

y_true, y_pred, all_images = [], [], []
for images, labels in test_ds_p:
    preds = cnn_model.predict(images, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())
    all_images.extend(images.numpy())

y_true = np.array(y_true)
y_pred = np.array(y_pred)

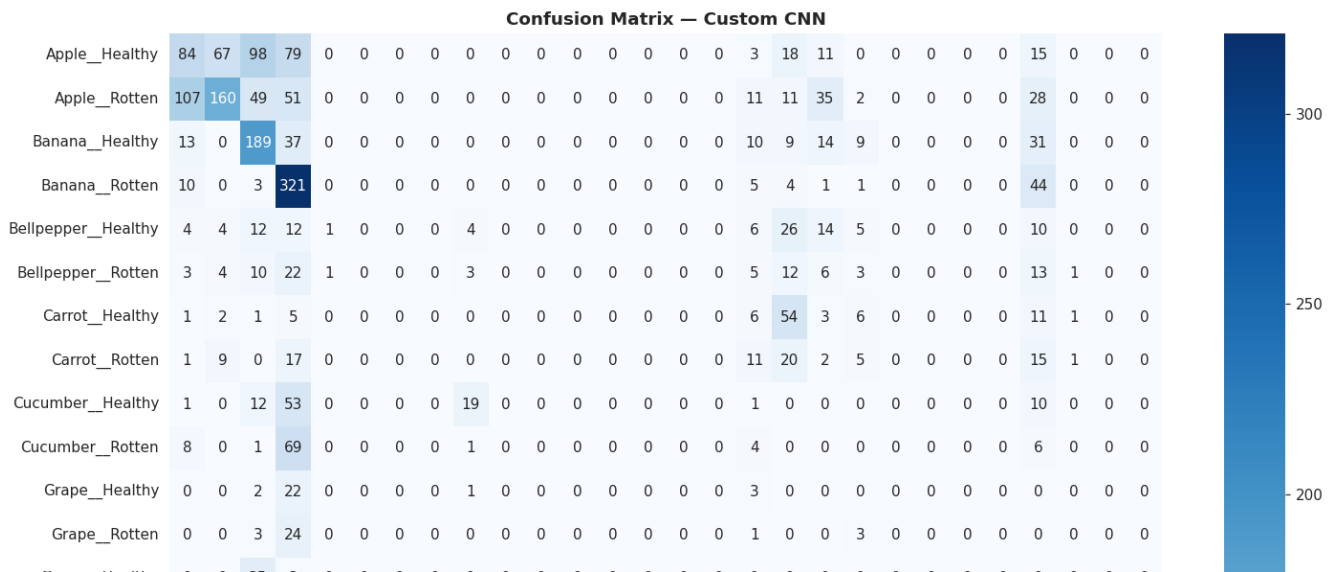
print(classification_report(y_true, y_pred, target_names=class_names, zero_division=0))

```

	precision	recall	f1-score	support
Apple_Healthy	0.26	0.22	0.24	375
Apple_Rotten	0.47	0.35	0.40	454
Banana_Healthy	0.41	0.61	0.49	312
Banana_Rotten	0.34	0.83	0.48	389
Bellpepper_Healthy	0.20	0.01	0.02	98
Bellpepper_Rotten	0.00	0.00	0.00	83
Carrot_Healthy	0.00	0.00	0.00	90
Carrot_Rotten	0.00	0.00	0.00	81
Cucumber_Healthy	0.50	0.20	0.28	96
Cucumber_Rotten	0.00	0.00	0.00	89
Grape_Healthy	0.00	0.00	0.00	28
Grape_Rotten	0.00	0.00	0.00	31
Guava_Healthy	0.00	0.00	0.00	38
Guava_Rotten	0.00	0.00	0.00	30
Jujube_Healthy	0.00	0.00	0.00	34
Jujube_Rotten	0.00	0.00	0.00	27
Mango_Healthy	0.15	0.16	0.15	275
Mango_Rotten	0.20	0.26	0.23	354
Orange_Healthy	0.55	0.75	0.63	319
Orange_Rotten	0.15	0.04	0.06	295
Pomegranate_Healthy	0.00	0.00	0.00	32
Pomegranate_Rotten	0.00	0.00	0.00	37
Potato_Healthy	0.00	0.00	0.00	95
Potato_Rotten	0.00	0.00	0.00	95
Strawberry_Healthy	0.21	0.86	0.33	242
Strawberry_Rotten	0.24	0.04	0.07	255
Tomato_Healthy	0.00	0.00	0.00	70
Tomato_Rotten	0.00	0.00	0.00	92
accuracy			0.31	4416
macro avg	0.13	0.15	0.12	4416
weighted avg	0.24	0.31	0.25	4416

Visualization 7: Confusion matrix

```
cm = confusion_matrix(y_true, y_pred)
fig, ax = plt.subplots(figsize=(max(8, 0.5*num_classes), max(7, 0.5*num_classes)))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names, yticklabels=class_names, ax=ax)
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_title("Confusion Matrix - Custom CNN", weight="bold")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



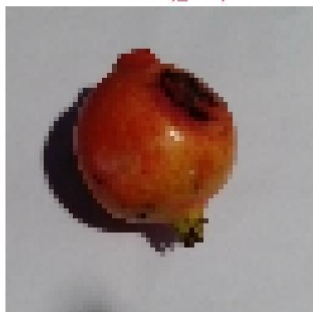
Visualization 8: Misclassified examples

```
mis_idx = np.where(y_true != y_pred)[0]
show_n = min(8, len(mis_idx))
sample_mis = np.random.choice(mis_idx, show_n, replace=False) if show_n > 0 else []

if show_n > 0:
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    for ax, idx in zip(axes.flatten(), sample_mis):
        ax.imshow(all_images[idx])
        ax.set_title(f"True: {class_names[y_true[idx]]}\nPred: {class_names[y_pred[idx]]}", fontsize=8, color="crimson")
        ax.axis("off")
    for ax in axes.flatten()[show_n:]:
        ax.axis("off")
    plt.suptitle("Misclassified Examples - Error Analysis", fontsize=14, weight="bold")
    plt.tight_layout()
    plt.show()
else:
    print("No misclassifications in this test batch sample - great result!")
```



True: Pomegranate_Rotten
Pred: Strawberry_Healthy



True: Banana_Healthy
Pred: Strawberry_Healthy



True: Apple_Healthy
Pred: Banana_Healthy



True: Apple_Rotten
Pred: Orange_Healthy

